

Sistemas Distribuidos y Programación Paralela

Trabajo Práctico 2

Sistemas Distribuidos y Concurrency

Informe

Grupo Cerberus

Salvador Baez (195157) · Mateo Nomico (168102) · Tomás Resnik (190168)

Universidad Nacional de Luján · Departamento de Ciencias Básicas

Cátedra: Dr. David Petrocelli · Comisión Mercedes · Segundo cuatrimestre de 2026

Lenguaje: Python 3.14 (FastAPI) · Repositorio: github.com/chimera-sdypp/sdypp_tp2 · Rama: main

Octubre de 2026

Contenido

1. Presentación
2. Hit #1 — Tareas remotas en contenedores
3. Hit #2 — Concurrencia y exclusión mutua
4. Hit #3 — Coordinación y tolerancia a fallos
5. Pruebas
6. CI/CD y despliegue público
7. Seguridad
8. Herramientas de IA
9. Conclusiones
10. Videos y anexo: cómo ejecutar

1. Presentación

El TP2 pide un servidor HTTP que resuelva tareas arbitrarias levantando un contenedor Docker por pedido, y hacerlo crecer en tres pasos: concurrencia con un pool de workers, una cola con exclusión mutua y relojes de Lamport (Hit #2), y varias instancias detrás de un balanceador con un coordinador elegido por el algoritmo Bully (Hit #3).

Los tres hits están en el repositorio, cada uno en su carpeta con un README que tiene las instrucciones de ejecución, los diagramas y las decisiones de diseño. Los tres corren en una VM de AWS EC2 con su health-check público:

Hit	Qué agrega	Health-check público
#1	Servidor + servicio tarea (imagen privada en Docker Hub)	http://18.231.127.74:8081/health
#2	Pool de N workers, cola con mutex, relojes de Lamport	http://18.231.127.74:8082/health
#3	3 nodos detrás de nginx, coordinador elegido con Bully	http://18.231.127.74:8083/health

Este informe resume cada hit, responde las preguntas del enunciado con las mediciones y agrega lo que atraviesa a los tres: pruebas, pipeline, despliegue y seguridad.

2. Hit #1 — Tareas remotas en contenedores

El cliente manda por POST un JSON con el cálculo, sus parámetros, datos adicionales y la imagen Docker que lo resuelve. El servidor (FastAPI, containerizado) valida el pedido, le pide al daemon de Docker que levante la imagen como contenedor temporal, espera a que su /health responda, le manda el trabajo a ejecutarTarea(), devuelve el resultado y borra el contenedor, salga bien o mal. El servicio tarea es otra imagen HTTP, cerberusdistribuido/tarea:1.0.0, que resuelve suma, resta, multiplicación y división.

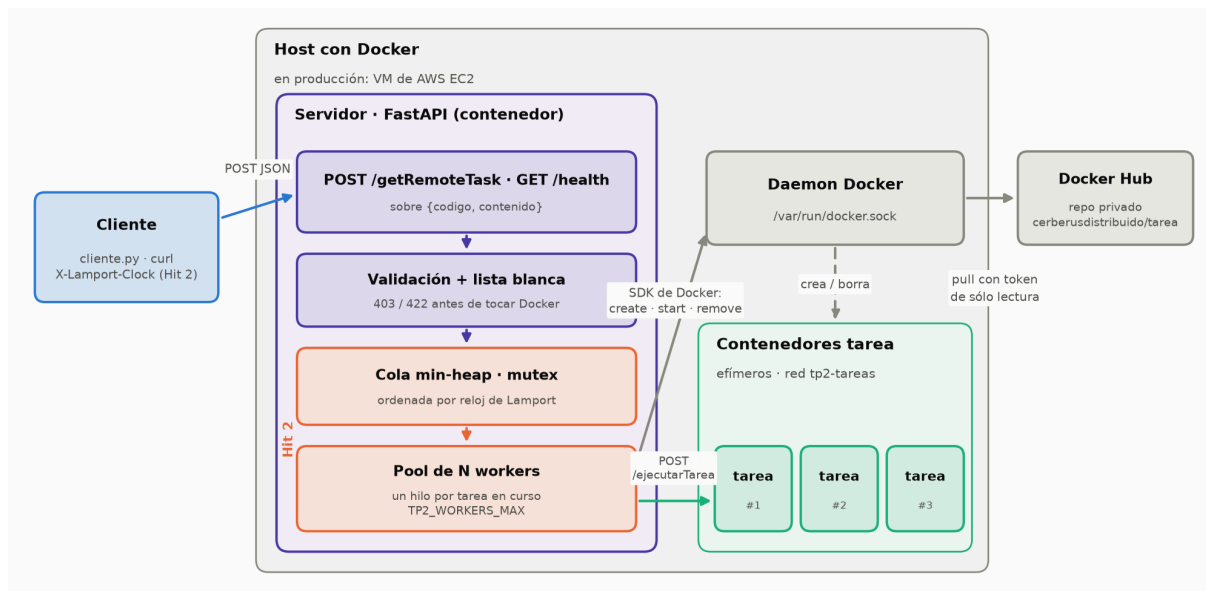


Figura 1. Arquitectura de los Hits #1 y #2. En naranja, lo que agrega el Hit #2.

Decisiones principales:

- **Lista blanca de imágenes.** El servidor tiene el socket de Docker, que equivale a root en el host: sin control, cualquiera ejecutaría lo que quisiera. Sólo se aceptan los repositorios de TP2 IMAGENES PERMITIDAS (403 si no) y siempre con una versión fija; latest se rechaza (422).

- **Un mismo sobre para todo.** Cada respuesta, buena o mala, es {codigo, contenido}; un error trae tipo, mensaje y detalles. Nada interno sale al cliente (IPs, nombres de contenedores, errores de Docker): eso va al log.
- **Health-check que no miente.** /health devuelve {servidor, docker} y contesta 503 si el daemon no responde, aunque el proceso esté vivo.
- **Contenedor efímero.** Se espera su /health en vez de un sleep fijo, se le habla por la red interna tp2-tareas sin publicar puertos y se borra siempre en un finally.

2.1 Credenciales del registry

La imagen de la tarea está en un repositorio privado de Docker Hub: un pull anónimo da "access denied". Elegimos configuración previa en el host: el servidor usa un token de acceso de Docker Hub de sólo lectura que el operador pone una sola vez en el .env del host (permisos 600). Compose se lo entrega al contenedor como archivo secreto en /run/secrets, no como variable de entorno, así que no aparece en docker inspect. El cliente nunca manda credenciales: un campo password en el payload es un 422.

Usuario y contraseña en el JSON	Token configurado en el host
Viaja en cada pedido y queda en logs, proxies y el historial del cliente	No sale del host
Lo conoce cada cliente: se filtra con el primero que se filtre	No lo conoce ningún cliente
Suele ser la contraseña: permite escribir y borrar imágenes	Token revocable y de sólo lectura
—	Se manda sólo a Docker Hub: un pull de otro registry no se lo lleva

Es el equivalente a un image pull secret de Kubernetes, sin Kubernetes. Un detalle que encontramos: hacer docker login en el host no alcanza, porque las credenciales de un pull las manda el cliente de Docker (acá, el SDK dentro del contenedor del servidor), no el daemon.

3. Hit #2 — Concurrencia y exclusión mutua

- **Pool de workers.** Como máximo TP2_WORKERS_MAX contenedores tarea a la vez. Cuando hay un lugar libre, el pool lanza un hilo para esa tarea; al terminar, ese hilo despacha la siguiente de la cola. Hilos y no procesos: el worker pasa casi todo el tiempo esperando a Docker, no usando CPU de Python.
- **Cola con exclusión mutua.** Una min-heap protegida por un threading.Lock. Encolar, despachar y liberar un worker se hacen con el lock tomado; el contenedor corre fuera del lock, así que una tarea lenta no frena a las demás.
- **Relojes de Lamport.** El reloj viaja en la cabecera X-Lamport-Clock. Al recibir, el servidor hace $\max(\text{propio}, \text{recibido}) + 1$, e incrementa una vez más al responder: si el cliente manda 500, la respuesta lleva 502, en la cabecera y en el cuerpo. La cola se ordena por el reloj del cliente: en la demo se mandaron tareas con relojes 300, 100 y 200 en ese orden, y se despacharon 100, 200, 300. Ordenar por el reloj del servidor al recibir daría el orden de llegada.

Por qué un mutex local y no Ricart-Agrawala: la cola y el contador de workers viven en la memoria de un solo proceso, y los que compiten son hilos de ese proceso. Comparten memoria, así que comparten el lock. Visto desde afuera es el algoritmo centralizado de exclusión mutua: el servidor es el coordinador que encola los pedidos y concede un worker cuando se libera. Un mutex distribuido haría falta si varias instancias compartieran la cola.

3.1 Throughput con 1, 2, 4 y 8 workers

Medido con cliente/benchmark.py: para cada N reinicia el servidor con TP2_WORKERS_MAX=N, le manda 32 tareas a la vez y repite tres veces. Todo en una misma máquina: Ryzen 5 3600 (6 núcleos / 12 hilos), 31 GB de RAM.

Workers	Tiempo para 32 tareas	Tareas/min	Speedup medido	Amdahl (P = 0,937)	Ideal
1	26,67 s ± 2,66	72	1,00x	1,00x	1x
2	15,02 s ± 2,81	128	1,78x	1,88x	2x
4	7,32 s ± 0,25	262	3,64x	3,36x	4x
8	4,88 s ± 0,12	393	5,46x	5,55x	8x

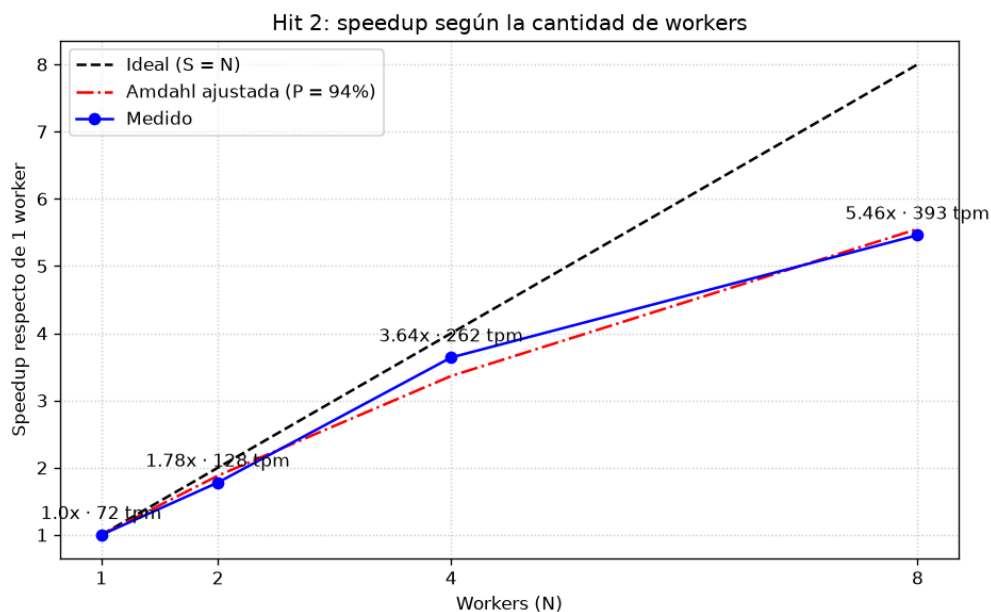


Figura 2. Speedup medido, ideal y la curva de Amdahl ajustada.

El speedup no es lineal: con 8 workers es 5,46x y no 8x. Ajustando la ley de Amdahl a lo medido, el 93,7 % del trabajo es paralelizable y el 6,3 % queda serializado; la curva predice 5,55x con 8 workers y el techo teórico sería $1 / (1 - 0,937) \approx 15,9x$. Con un worker, cada tarea cuesta ~0,83 s y casi todo es el ciclo de vida del contenedor (crear, arrancar, esperar su /health, borrar): la suma tarda microsegundos.

3.2 Cuellos de botella en un solo equipo

Lo que se midió durante el benchmark (top cada ~3 s, agrupado por contenedores tarea corriendo a la vez):

Contenedores a la vez	CPU total	dockerd	containerd + shims
1	18 %	11 %	5 %
2	29 %	29 %	15 %
3-4	52 %	59 %	28 %
5-8	79 %	93 %	46 %

El primer cuello de botella es el daemon de Docker: crear, arrancar y borrar contenedores pasa por un único dockerd, que con 5 a 8 contenedores ocupa casi un núcleo entero. Cómo se identificaría cada recurso compartido:

Recurso	Por qué limita	Cómo se mide
Daemon de Docker	Un solo proceso con locks internos para red, cgroups y capas	top / pidstat sobre dockerd; latencia de containers.create(); docker events
CPU	Cada tarea arranca su propio intérprete de Python	top, mpstat -P ALL 1
Red de Docker	Una interfaz veth y una IP nueva por contenedor	ip link, netstat -s
I/O de disco	Capas overlay2 y logs en disco	iostat -xz 1, iotop
Memoria	~30–50 MB por contenedor tarea	docker stats, free -m

4. Hit #3 — Coordinación y tolerancia a fallos

Tres instancias del servidor del Hit #1 detrás de nginx (round-robin). Todas ejecutan tareas; la de ID más alto que esté viva es además el coordinador: decide qué nodo ejecuta cada tarea (el vivo con menos tareas en curso) y lleva el registro de estado de los nodos, que arma con un heartbeat por segundo de cada uno. El nodo que recibió el pedido es su dueño hasta que termina: el coordinador sólo decide a quién va.

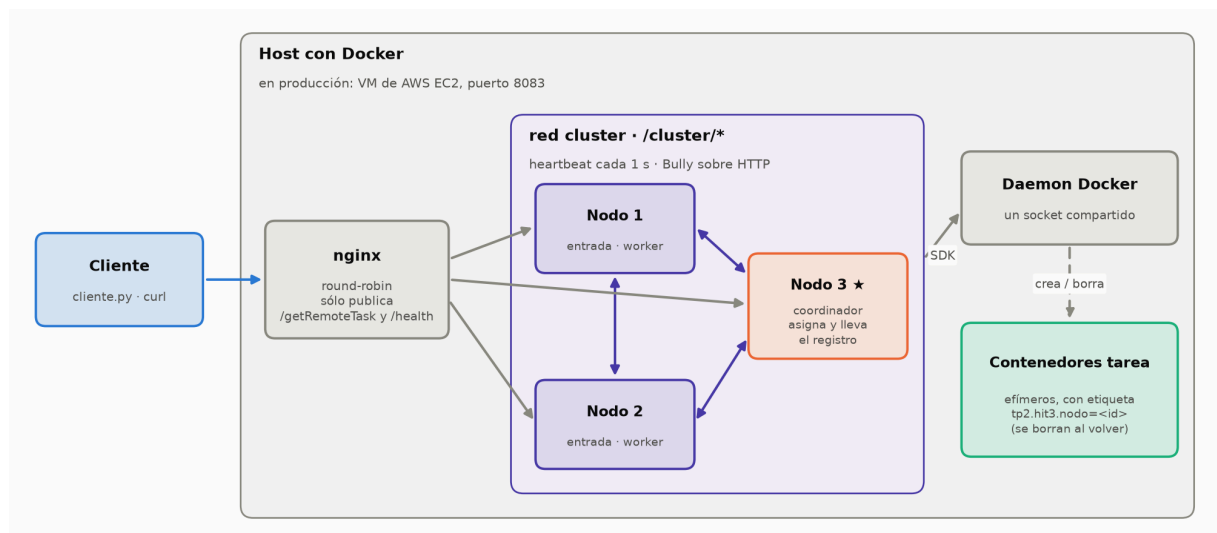


Figura 3. Arquitectura del Hit #3. /cluster/* no se publica por nginx.

4.1 Elección de líder con Bully

Gana el nodo vivo de ID más alto. Un nodo convoca una elección al arrancar, cuando el heartbeat al coordinador falla o cuando el coordinador no contesta un pedido de asignación. Manda ELECCION a los de ID mayor; si alguno contesta OK, espera su COORDINADOR; si nadie contesta, gana y se anuncia a todos. La Figura 4 es una caída real del coordinador, con los tiempos de los logs.

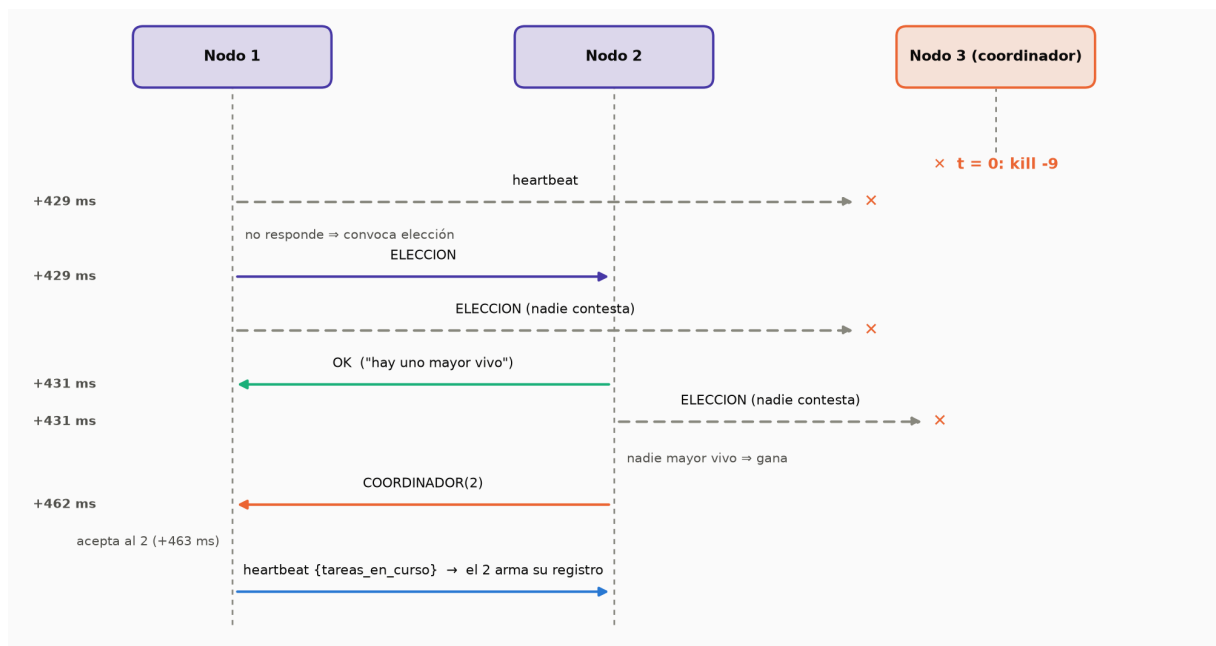


Figura 4. Diagrama de secuencia: se mata al nodo 3 (coordinador) y gana el 2.

Cuando el nodo 3 vuelve, convoca su elección de arranque, nadie es mayor y recupera el puesto en ~1 s.

4.2 Tiempo de recuperación

Cinco caídas del coordinador por escenario (docker kill, SIGKILL), con los instantes tomados de los logs de los nodos. Media (mínimo–máximo) en ms desde el kill:

Escenario	Detección	Elección	Líder nuevo	Lo ve un cliente
Con carga (3 clientes)	1 (0–3)	2 (1–4)	4 (1–6)	14 (11–17)
Sin carga (sólo heartbeat)	599 (156–851)	28 (23–34)	627 (185–885)	635 (195–889)

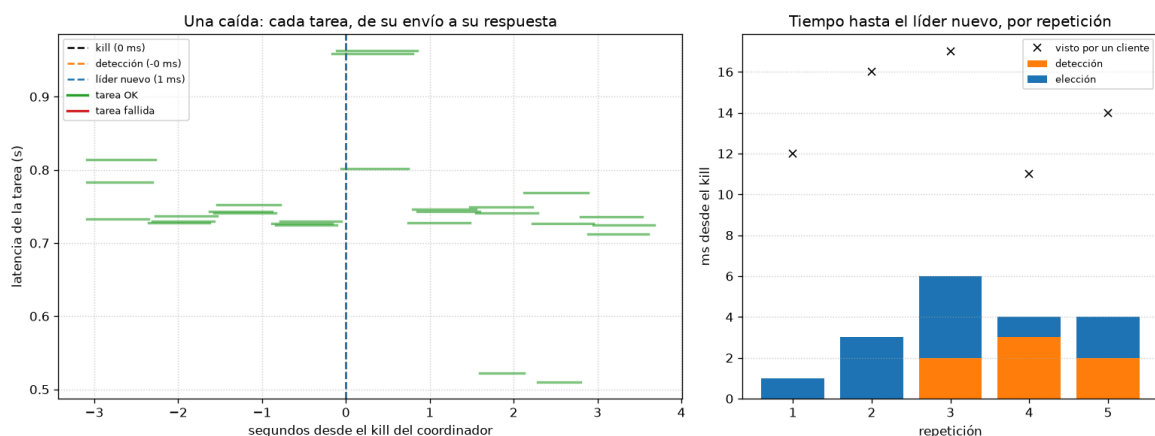


Figura 5. Con carga: las tareas alrededor de una caída (izquierda) y el tiempo hasta el líder nuevo en cada repetición (derecha).

Casi todo el tiempo es la detección; la elección cuesta unos pocos ms. Con carga, alguna tarea en curso nota la conexión cortada al instante. Sin carga la detecta el heartbeat, así que depende de en qué punto del intervalo de 1 s cae el kill; el peor caso es el intervalo más su timeout (2 s) si el nodo se cuelga sin morir.

4.3 Redistribución de las tareas pendientes

Qué se cae	Qué pasa con las tareas
El coordinador, con tareas pidiendo asignación	El nodo de entrada convoca la elección y reintenta con el coordinador nuevo (hasta 10 s).
El coordinador, con tareas ejecutándose en otros nodos	Nada: el coordinador no está en ese camino.
Un nodo que ejecutaba una tarea	El nodo de entrada pide otra asignación excluyéndolo y la tarea se re-ejecuta en otro.
El nodo de entrada	Esa tarea se pierde (502): nginx no reintenta un POST. El cliente puede reintentar.

En la medición con carga, las cinco caídas afectaron 23 tareas: 16 terminaron bien (4 reasignadas desde el nodo caído) y 7 fallaron con 502, las que tenían al nodo caído como entrada. Cerrar ese caso requiere que el cliente reintente o una cola replicada. Los contenedores tarea que quedan huérfanos se borran cuando el nodo vuelve.

5. Pruebas

Cada hit tiene pruebas unitarias y de API que no necesitan Docker, pruebas de integración que levantan todo con Docker real, y las del servicio tarea:

Hit	Unitarias y de API	Integración	Servicio tarea	Total
Hit 1	110	8	11	129
Hit 2	117	8	11	136
Hit 3	168	15	11	194
Total				459

- Las de integración usan una tarea de prueba que permite forzar cada falla (división por cero, error 500, timeout) y al final corren el cliente contra la tarea real.
- En el Hit #3, la integración mata al coordinador y a un ejecutor con una tarea en curso, y comprueba que las tareas siguen y que el nodo que vuelve borra sus contenedores huérfanos.
- Bully se prueba también sin red: tres instancias conectadas en memoria, con caídas y casos borde.

6. CI/CD y despliegue público

Cada push corre en GitHub Actions gitleaks y, por hit, los tests unitarios y los de integración. En main, además, publica la imagen de cada servidor en GitHub Container Registry con el GITHUB_TOKEN efímero del job.

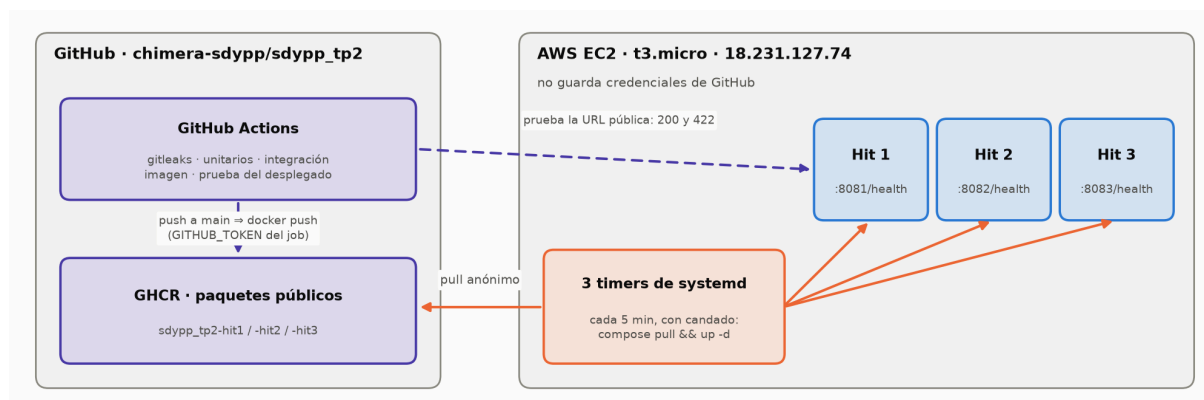


Figura 6. Pipeline y despliegue en la VM de AWS EC2.

- La VM no guarda credenciales de GitHub ni GitHub de la VM: un timer de systemd por hit hace docker compose pull && up -d cada 5 minutos contra paquetes públicos. Los tres timers comparten un candado (flock), porque el image prune de uno cortaba la descarga de otro.
- Después del despliegue, un job por hit prueba la URL pública: una suma (200) y una división por cero (422), que incluye el pull de la imagen privada de la tarea desde la VM.
- Logs en tres destinos: consola (docker compose logs), archivo rotativo en un volumen y un buffer en memoria con las últimas 500 líneas.

7. Seguridad

- .gitignore desde el inicio; no se versionan .env ni credenciales, sólo .env.example.
- gitleaks en el pipeline: si detecta un secret, falla.
- El token de Docker Hub es de sólo lectura, vive en el .env del host (600) y llega como archivo secreto, no por el payload (sección 2.1).
- Lista blanca de imágenes y versión fija obligatoria: el servidor no ejecuta imágenes arbitrarias.
- Los servidores corren con un usuario sin privilegios. El socket de Docker sí se monta, porque es la forma de crear las tareas; por eso la lista blanca.
- En el Hit #3, /cluster/* no se publica: nginx sólo expone /getRemoteTask y /health. Los mensajes entre nodos no se autentican; los protege que la red es interna.
- No hay OIDC / Workload Identity porque el pipeline no se autentica contra el proveedor de nube: la VM trae la imagen sola. Corresponde cuando el despliegue lo haga el pipeline (TP3).

8. Herramientas de IA

Claude

Lo usó Tomás. Ayudó a:

- Revisar el servidor del Hit #1, y escribir el servicio tarea y el cliente.
- Investigar cómo autenticar el pull contra un registry privado sin pasar credenciales por el payload.
- Armar el pipeline (gitleaks, tests, integración, imágenes) y el verificar el despliegue.
- Extender el contrato de la API (OpenAPI), los README y editar los videos.

Claude (Claude Code)

Lo usó Mateo. Ayudó a:

- Extender el servidor del Hit #1 a un cluster de 3 nodos detrás de nginx, con la elección de coordinador por Bully, el registro de nodos por heartbeats y la reasignación de tareas cuando se cae un nodo.
- Escribir los tests unitarios y los de integración contra Docker real (kill del coordinador y de un ejecutor con una tarea en curso).
- Medir el tiempo de recuperación ante la caída del coordinador, con y sin carga, y armar los gráficos.
- Armar el pipeline del Hit #3 (tests, integración, imagen en GHCR) y el despliegue en la VM.

Antigravity

Lo usó Salvador. Ayudó a:

- Extender el servidor del Hit #1 con un pool de workers, exclusión mutua sobre los recursos compartidos y relojes de Lamport para ordenar las tareas.

- Escribir los tests unitarios (pool, Lamport, validación, lanzador, API) y los de integración contra Docker.
- Escribir el servicio tarea y el cliente de benchmark, medir el throughput y armar el gráfico de escalabilidad.
- Armar el pipeline del Hit #2 (CI/CD) y el script de instalación en la VM.

9. Conclusiones

- Levantar un contenedor por tarea aísla bien, pero cuesta ~0,8 s por tarea y concentra el trabajo en el daemon de Docker: es lo que limita el speedup a 5,46x con 8 workers, no la cola ni el mutex.
- La exclusión mutua hay que elegirla según quién comparte qué: un lock local alcanza dentro de un proceso; un mutex distribuido sólo tiene sentido si varias instancias comparten la cola.
- El reloj de Lamport sólo aporta si ordena por el evento de envío: ordenar por el reloj del receptor reproduce el orden de llegada.
- En Bully, casi todo el tiempo de recuperación es detectar la caída, no elegir. Que cada tarea la siga su nodo de entrada y no el coordinador hace que la caída del coordinador no pierda tareas.
- Las credenciales del registry no tienen que pasar por el cliente: un token de sólo lectura configurado en el host es más seguro y más simple.

10. Videos y anexo: cómo ejecutar

Videos

Hay un video por hit en el repositorio, enlazado desde el README de su carpeta.

Ejecución desde la terminal

Cada hit se levanta desde su carpeta, sin IDE. Hace falta Docker con Compose v2 y Python 3 para el cliente.

```
git clone https://github.com/chimera-sdypp/sdypp_tp2 && cd sdypp_tp2/hit1 # o hit2, hit3
cp .env.example .env # DOCKER_GID, TP2_IMAGENES_PERMITIDAS y el token de sólo lectura
docker compose up --build -d --wait
curl -s localhost:8080/health
python3 cliente/cliente.py suma '{"a": 3, "b": 4}' --imagen cerberusdistribuido/tarea:1.0.0

# Hit 2: medir el throughput con 1, 2, 4 y 8 workers
python3 cliente/benchmark.py

# Hit 3: matar al coordinador y ver la elección
docker compose kill nodo3 && curl -s localhost:8080/health
docker compose logs nodo1 nodo2 | grep bully

# Tests de un hit
cd servidor && pip install -r requirements-dev.txt && python -m pytest && ./tests/integracion.sh
```

Referencias

- Enunciado del TP2: <https://dpetrocelli.github.io/sd2026/practica-2.html>
- Repositorio: https://github.com/chimera-sdypp/sdypp_tp2 (contrato de la API en contrato-publico.md y openapi.yaml)

- [AMD67] Amdahl, G. M. (1967). Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities. AFIPS Conference Proceedings, 30, 483–485.
- [GAR82] Garcia-Molina, H. (1982). Elections in a Distributed Computing System. IEEE Transactions on Computers, C-31(1), 48–59.
- [LAM78] Lamport, L. (1978). Time, Clocks, and the Ordering of Events in a Distributed System. Communications of the ACM, 21(7), 558–565.
- [RIC81] Ricart, G. y Agrawala, A. K. (1981). An Optimal Algorithm for Mutual Exclusion in Computer Networks. Communications of the ACM, 24(1), 9–17.